Capability-Bounded Code Evolution with LLMs

This repository contains the public research artifact for Fabio Cozzuto's MSc thesis at Bishop's University:

Capability-Bounded Code Evolution with Large Language Models

The project studies whether large language models can iteratively improve executable heuristic programs when they receive benchmark feedback. The work uses a staged empirical record across simple adversarial games, symmetric and asymmetric traveling salesperson benchmarks, capacitated vehicle routing benchmarks, bounded real-world CVRP instances, model-strength comparisons, and a state-of-the-art CVRP calibration based on PyVRP's hybrid genetic search implementation.

The retained thesis claim is bounded: LLM-guided code evolution can produce measurable improvements under some feedback and replay conditions, while strong domain-specific algorithms continue to define the performance limits in harder routing settings. The repository is organized so the reported aggregate evidence can be inspected without rerunning paid API campaigns.

Repository Scope

This repository is the public research artifact for the thesis. It contains experimental harnesses, benchmark preparation scripts, archived run outputs, aggregate reports, and protocol documents.

Some archived runs contain prompt text, model responses, candidate JSON files, generated code, validation records, and fallback/error records. These files are experiment records for the code-evolution runs and provide provenance for the aggregate reports.

The public artifact excludes credentials, local environment files, private thesis-revision notes, large transient worker outputs, and unpublished presentation material.

Artifact Map

ARTIFACT_MANIFEST.md maps each thesis phase to the corresponding protocol documents, configuration files, run archives, aggregate reports, and version tags.

Tagged phase archives and committed run reports are the reference points for reported evidence. Branch names are working labels, not the empirical record.

Repository Layout

- configs/ contains experiment configurations for the staged campaigns.
- docs/ contains public protocol documents, phase reports, and analysis notes.
- runs/ contains archived official run outputs and selected generated artifacts.
- benchmarks/ contains benchmark preparation material and local benchmark documentation.
- llm_grid_battle/, llm_tsp/, llm_atsp/, llm_cvrp/, llm_cvrp_phase9/, llm_tsp_operator/, and llm_tsp_portfolio/ contain the experiment harnesses and supporting code.
- Top-level run_*, aggregate_*, prepare_*, and analyze_* scripts run the official campaigns, rebuild benchmark manifests, and regenerate aggregate evidence from archived outputs.
- run_cvrp_phase12_sota_hgs_baseline.py runs the PyVRP HGS-style CVRP calibration used as the final state-of-the-art baseline check.

Reproducibility Paths

The reported claims can be inspected at different levels of effort.

1. No execution: read the protocol documents, phase reports, run summaries, and artifact manifest.
2. No paid API calls: install the local dependencies and re-run aggregation, plotting, or baseline scripts against archived outputs.
3. External solver reruns: install the optional PyVRP/VRPLIB dependencies and rerun the Phase 12 HGS-style CVRP calibration.
4. Full LLM campaign reruns: provide API credentials and rerun selected evolution campaigns. This path incurs API costs. The archived aggregate tables do not require paid model access for inspection.

The public artifact is intended to make the reported aggregate claims auditable without requiring paid model access.

Setup

Create an isolated Python environment and install the dependencies:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The core dependencies support local analysis and report generation. To rerun the Phase 12 PyVRP HGS-style CVRP calibration, install the optional dependency file:

```
python -m pip install -r requirements-phase12.txt
```

API credentials are only needed for new LLM campaign reruns. Keep credentials in local environment variables or ignored .env files. Do not commit credentials.

Official Phase Tags

The following tags identify the archived code states used during the project:

- phase-1-baseline
- phase-2-adversarial-curriculum
- phase-2b-factorial-holdout
- phase-3-transfer
- phase-4-causal-transfer
- phase-5-routing-archive
- phase-6-replay-mechanism
- phase-7-operator-discovery
- phase-8-adaptive-portfolio
- phase-9-real-world-vrp
- phase-9-closeout-budget-control
- phase-10-cross-family-synthesis
- phase-11-model-strength-factorial
- phase-12-sota-cvrp-hgs-baseline

If a branch has moved after a phase, the corresponding tag and archived run directory identify the phase evidence.

Citation

Citation metadata is provided in CITATION.cff. If you use this artifact, cite the repository together with the thesis or any later publication that supersedes it.